Chapter-3

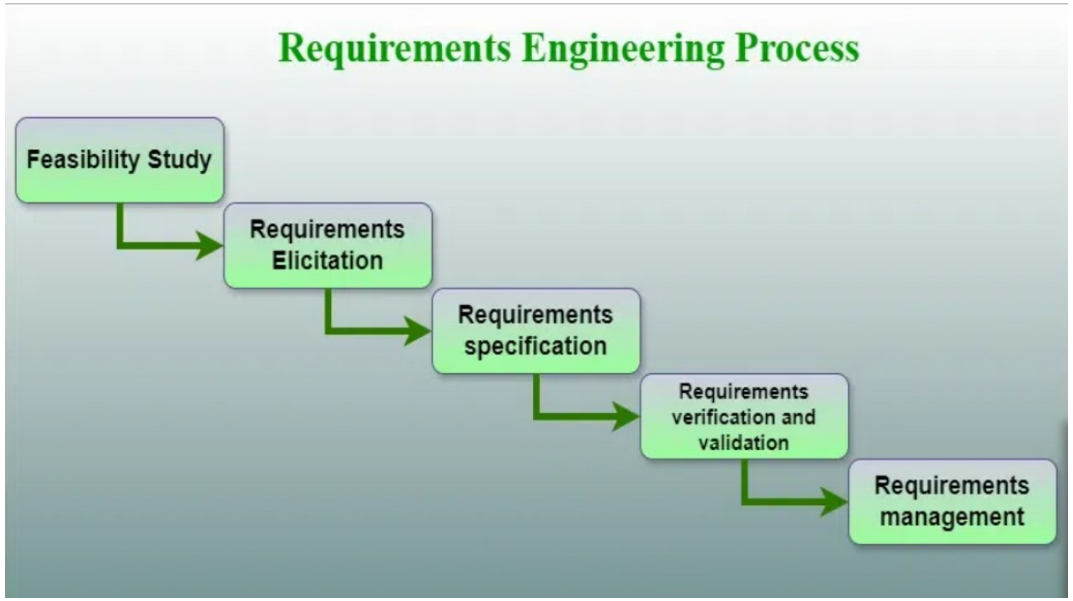
Requirement Elicitation

An overview of requirements elicitation.

- **Requirements Engineering** is the process of identifying, eliciting, analyzing, specifying, validating, and managing the needs and expectations of stakeholders for a software system.
- A systematic and strict approach to the definition, creation, and verification of requirements for a software system is known as requirements engineering.

Cont’d

- To guarantee the effective creation of a software product, the requirements engineering process entails several tasks that help in understanding, recording, and managing the demands of stakeholders.



3.1. Requirement engineering process

- **Feasibility Study**
- **Requirements elicitation**
- **Requirements specification**
- **Requirements for verification and validation**
- **Requirements management**

3.1.1. Feasibility Study

The feasibility study mainly concentrates on below five mentioned areas below. Among these Economic Feasibility Study is the most important part of the feasibility analysis and the Legal Feasibility Study is less considered feasibility analysis.

- ✓ Technical Feasibility
- ✓ Operational Feasibility
- ✓ Economic Feasibility
- ✓ Legal Feasibility

3.1.2. Requirements Elicitation

- It is related to the various ways used to gain knowledge about the project domain and requirements.
- The various sources of domain knowledge include customers, business manuals, the existing software of the same type, standards, and other stakeholders of the project.
- The techniques used for requirements elicitation include interviews, brainstorming, task analysis, Delphi technique, prototyping, etc.
- Elicitation does not produce formal models of the requirements understood. Instead, it widens the domain knowledge of the analyst and thus helps in providing input to the next stage.

Cont’d

- Requirements elicitation is the process of gathering information about the needs and expectations of stakeholders for a software system.
- This is the first step in the requirements engineering process and it is critical to the success of the software development project.
- The goal of this step is to understand the problem that the software system is intended to solve and the needs and expectations of the stakeholders who will use the system.

Cont’d

- Several techniques can be used to elicit requirements, including:
- **Interviews**: These are one-on-one conversations with stakeholders to gather information about their needs and expectations.
- **Surveys**: These are questionnaires that are distributed to stakeholders to gather information about their needs and expectations.
- **Focus Groups**: These are small groups of stakeholders who are brought together to discuss their needs and expectations for the software system.
- **Observation**: This technique involves observing the stakeholders in their work environment to gather information about their needs and expectations.

Cont’d

- **Prototyping**: This technique involves creating a working model of the software system, which can be used to gather feedback from stakeholders and to validate requirements.
- It’s important to document, organize, and prioritize the requirements obtained from all these techniques to ensure that they are complete, consistent, and accurate.

3.1.3. Requirements Specification

- This activity is used to produce formal software requirement models. All the requirements including the functional as well as the non-functional requirements and the constraints are specified by these models in totality.
- During specification, more knowledge about the problem may be required which can again trigger the elicitation process.
- The models used at this stage include ER diagrams, data flow diagrams(DFDs), function decomposition diagrams(FDDs), data dictionaries, etc.

Cont’d

- Requirements specification is the process of documenting the requirements identified in the analysis step in a clear, consistent, and unambiguous manner.
- This step also involves prioritizing and grouping the requirements into manageable chunks.
- The goal of this step is to create a clear and comprehensive document that describes the requirements for the software system. This document should be understandable by both the development team and the stakeholders.

Cont’d

- **Several types of requirements are commonly specified in this step, including**
- **Functional Requirements:** These describe what the software system should do. They specify the functionality that the system must provide, such as input validation, data storage, and user interface.
- **Non-Functional Requirements**: These describe how well the software system should do it. They specify the quality attributes of the system, such as performance, reliability, usability, and security.
- **Constraints:** These describe any limitations or restrictions that must be considered when developing the software system.
- **Acceptance Criteria**: These describe the conditions that must be met for the software system to be considered complete and ready for release.

Cont’d

- To make the requirements specification clear, the requirements should be written in a natural language and use simple terms, avoiding technical jargon, and using a consistent format throughout the document.
- It is also important to use diagrams, models, and other visual aids to help communicate the requirements effectively.
- Once the requirements are specified, they must be reviewed and validated by the stakeholders and development team to ensure that they are complete, consistent, and accurate.

3.1.4. Requirements Verification and Validation

- **Verification:** It refers to the set of tasks that ensures that the software correctly implements a specific function.
- **Validation:** It refers to a different set of tasks that ensures that the software that has been built is traceable to customer requirements. If requirements are not validated, errors in the requirement definitions would propagate to the successive stages resulting in a lot of modification and rework.

Cont’d

steps for this process include:

- The requirements should be **consistent** with all the other requirements i.e. no two requirements should conflict with each other.
- The requirements should be **complete** in every sense.
- The requirements should be practically achievable.
- Reviews, buddy checks, making test cases, etc. are some of the methods used for this.
- Requirements verification and validation (V&V) is the process of checking that the requirements for a software system are complete, consistent, and accurate and that they meet the needs and expectations of the stakeholders.

Cont’d

- The goal of V&V is to ensure that the software system being developed meets the requirements and that it is developed on time, within budget, and to the required quality.
- Verification is checking that the requirements are complete, consistent, and accurate. It involves reviewing the requirements to ensure that they are clear, testable, and free of errors and inconsistencies.
- This can include reviewing the requirements document, models, and diagrams, and holding meetings and walkthroughs with stakeholders.

Cont’d

- Validation is the process of checking that the requirements meet the needs and expectations of the stakeholders.
- It involves testing the requirements to ensure that they are valid and that the software system being developed will meet the needs of the stakeholders.
- This can include testing the software system through simulation, testing with prototypes, and testing with the final version of the software.

Cont’d

- Verification and Validation is an iterative process that occurs throughout the software development life cycle.
- It is important to involve stakeholders and the development team in the V&V process to ensure that the requirements are thoroughly reviewed and tested.
- It’s important to note that V&V is not a one-time process, but it should be integrated and continue throughout the software development process and even in the maintenance stage.

3.1.5. Requirements Management

- Requirement management is the process of analyzing, documenting, tracking, prioritizing, and agreeing on the requirement and controlling the communication with relevant stakeholders.
- This stage takes care of the changing nature of requirements. It should be ensured that the SRS is as modifiable as possible to incorporate changes in requirements specified by the end users at later stages too.
- Modifying the software as per requirements in a systematic and controlled manner is an extremely important part of the requirements engineering process.

Cont’d

- Requirements management is the process of managing the requirements throughout the software development life cycle, including tracking and controlling changes, and ensuring that the requirements are still valid and relevant.
- The goal of requirements management is to ensure that the software system being developed meets the needs and expectations of the stakeholders and that it is developed on time, within budget, and to the required quality.

3.2. Software Lifecycle Definition

- Software lifecycle

  Models for the development of software

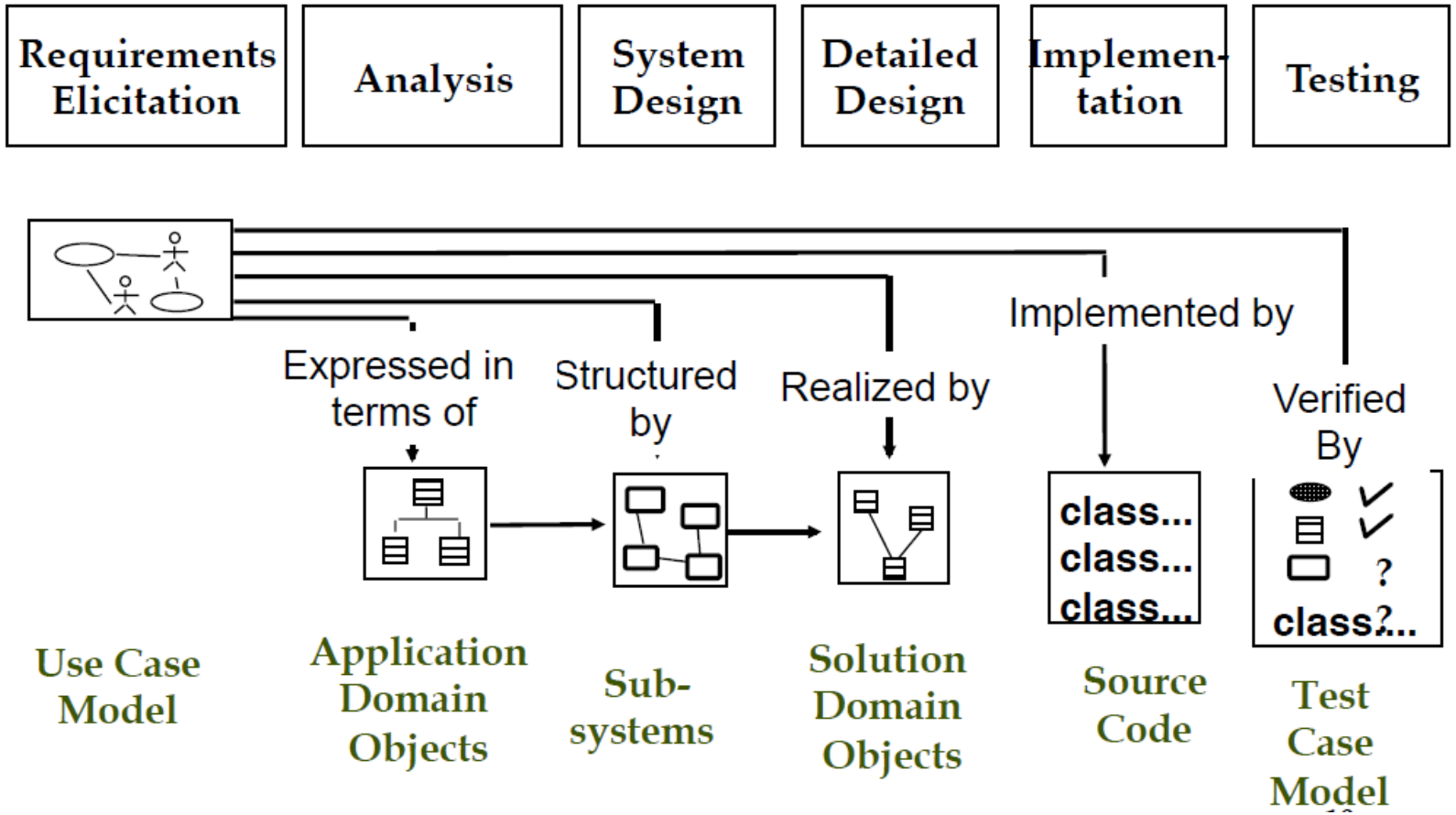
Set of **activities and** their **dependency relationship**s to each other to support the development of a software system

- Examples:

Analysis, design, implementation, testing.

Design depends on analysis, testing can be done before implementation

Software Lifecycle Activities and and their models



First step in identifying the Requirements: System identification

Two questions need to be answered:

1. How can we identify the purpose of a system?

What are the requirements, what are the constraints?

2. What is inside, what is outside the system?

These two questions are answered during

requirements elicitation and analysis

- Requirements elicitation:

  Definition of the system in terms understood by the customer and/or user ("Requirements specification")

- Analysis:

Definition of the system in terms understood by the developer (Technical specification, "Analysis model")

- Requirements Process: Consists of the activities Requirements Elicitation and Analysis.

Techniques to elicit Requirements

- Bridging the gap between end user and developer:
- **Questionnaires:** Asking the end user a list of preselected questions
- **Task Analysis:** Observing end users in their operational environment
- **Scenarios:** Describe the use of the system as a series of interactions between a specific end user and the system
- **Use cases:** Abstractions that describe a class of scenarios.

Scenarios

Scenario:-

- A synthetic description of an event or series of actions and events
- A textual description of the usage of a system.
- The description is written from an end user's point of view
- A scenario can include text, video, pictures and story boards.
- It usually also contains details about the workplace, social situations and resource constraints.
- "A narrative description of what people do and experience as they try to make use of computer systems and applications"
- [M. Carroll, Scenario-Based Design, Wiley, 1995]

After the scenarios are formulated

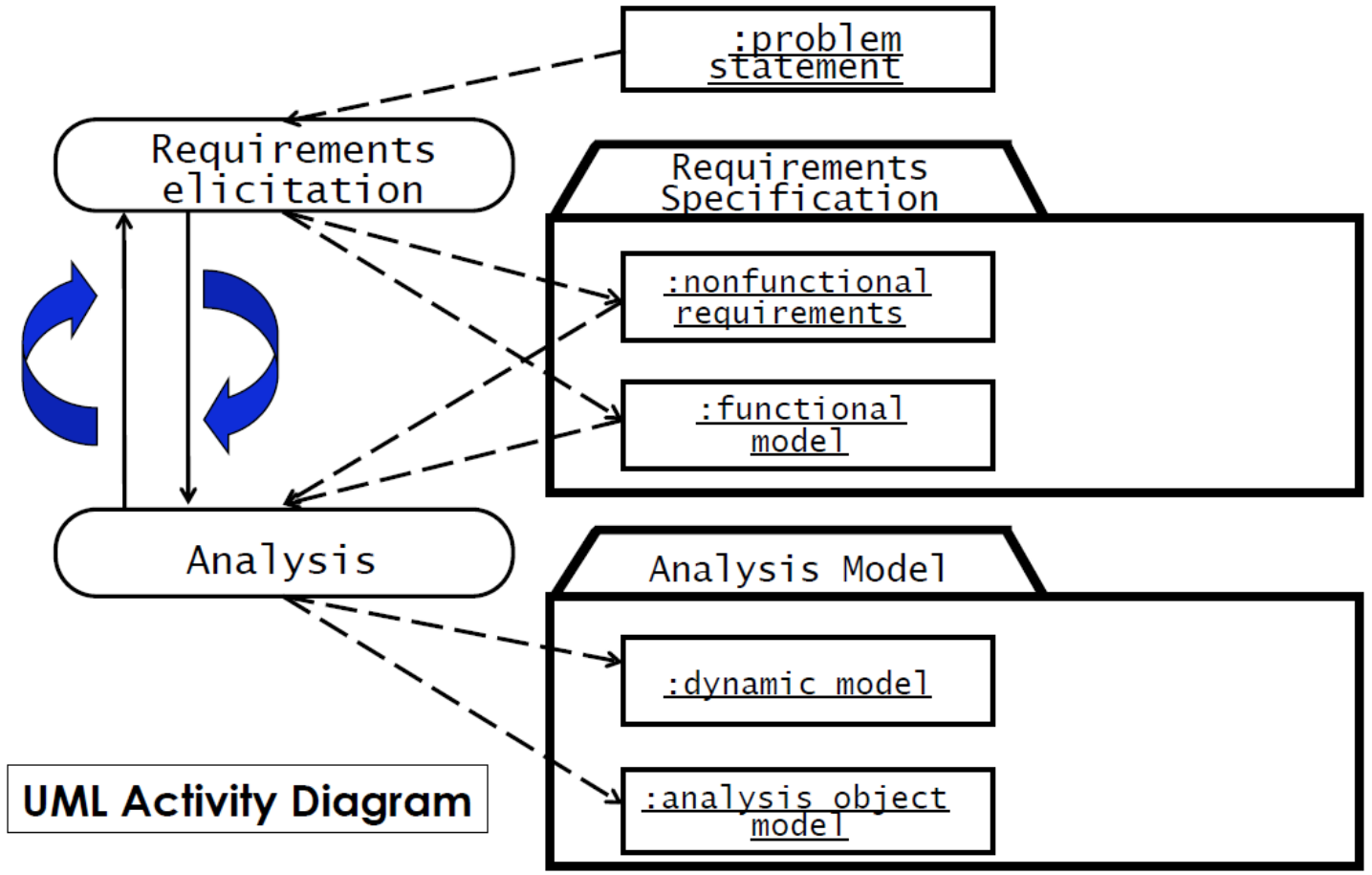
- Find all the use cases in the scenario that specify all instances of how to report a fire

Example from the Warehouse on Fire scenario:

- "Report Emergency" is a candidate for a use case
- Describe each of these use cases in more detail
  - ✓ Participating actors
  - ✓ Describe the entry condition
  - ✓ Describe the flow of events
  - ✓ Describe the exit condition
  - ✓ Describe exceptions
  - ✓ Describe nonfunctional requirements

The set of all use cases is the basis for the Functional Model

Requirements Process



Requirements Specification vs Analysis Model

- Both are models focusing on the requirements from the user's view of the system

The requirements specification uses natural language (derived from the problem statement)

- The analysis model uses a formal or semi-formal notation
  - ✓Requirements Modeling Languages
  - ✓Natural Language
  - ✓Graphical Languages: UML, SysML, SA/SD
  - ✓Mathematical Specification Languages: VDM (ViennaDefinition Method), Z (based on Zermelo–Fraenkel set theory), Formal methods ….

Types of Requirements

- Functional requirements

Describe the interactions between the system and its environment independent from the implementation

i.e. “An operator must be able to define a new game“

- Nonfunctional requirements

Aspects not directly related to functional behavior

i.e. “The response time must be less than 1 second”

- Constraints

Imposed by the client or the environment

i.e.“The implementation language must be Java “

- Also called “Pseudo requirements”.

Functional vs. Nonfunctional Requirements

Functional Requirements

- Describe user tasks which the system needs to support
- Phrased as actions

✓"Advertise a new league"

✓"Schedule tournament"

✓"Notify an interest group"

Nonfunctional Requirements

- Describe properties of the system or the domain
- Phrased as constraints or negative assertions
- "All user inputs should be acknowledged within 1 second"
- "A system crash should not result in data loss".

Types of Nonfunctional Requirements

Quality requirements

- Usability
- Reliability
  - Robustness
  - Safety
- Performance
  - Response time
  - Scalability
  - Throughput
  - Availability
- Supportability
  - Adaptability
  - Maintainability

Constraints or Pseudo requirements

- Implementation
- Interface
- Operation
- Packaging
- Legal
  - Licensing (GPL, LGPL)
  - Certification
  - Regulation

Some Quality Requirements Definitions

- **Usability**

The ease with which actors can perform a function in a system

Usability is one of the most frequently misused terms ("The system is easy to use")

Usability must be *measurable*, otherwise it is *marketing*

Example: Specification of the number of steps – the measure! -

to perform a internet-based purchase with a web browser

Robustness: The ability of a system to maintain a function

- ✓ even if the user enters a wrong input
- ✓ even if there are changes in the environment

Example: The system can tolerate temperatures up to 90 C

Availability: The ratio of the expected uptime of a system to the aggregate of the expected up and down time

Example: The system is down not more than 5 minutes per week.

Nonfunctional Requirements: Examples

- “Spectators must be able to watch a match without prior registration and without prior knowledge of the match.”

  Usability Requirement

- “The system must support 10 parallel tournaments”

  Performance Requirement

What should not be in the Requirements?

- System structure, implementation technology
- Development methodology
- Development environment
- Implementation language
- Reusability
- It is desirable that none of these above are constrained by the client.

Requirements Validation

- Requirements validation is a quality assurance step, usually performed after requirements elicitation or after analysis
- Correctness:

  The requirements represent the client's view
- Completeness:

  All possible scenarios, in which the system can be used,are described
- Consistency:

  There are no requirements that contradict each other.

Requirements Validation (2)

- Clarity:
  Requirements can only be interpreted in one way
- Realism:
  Requirements can be implemented and delivered
- Traceability:
  Each system component and behavior can be traced to a set of functional requirements
- Problems with requirements validation:
  - ✓ **Requirements change quickly** during requirements elicitation
  - ✓ Inconsistencies are easily added with each change
  - ✓Tool support is needed!

Requirements Analysis Document Template

1. Introduction
2. Current system
3. Proposed system
   - 3.1 Overview
   - 3.2 Functional requirements
   - 3.3 Nonfunctional requirements
   - 3.4 Constraints (“Pseudo requirements”)
   - 3.5 System models
     - 3.5.1 Scenarios
     - 3.5.2 Use case model
     - 3.5.3 Object model
       - 3.5.3.1 Data dictionary
       - 3.5.3.2 Class diagrams
     - 3.5.4 Dynamic models
     - 3.5.5 User interface
4. Glossary

Section 3.3 Nonfunctional Requirements

3.3.1 User interface and human factors
3.3.2 Documentation
3.3.3 Hardware considerations
3.3.4 Performance characteristics
3.3.5 Error handling and extreme conditions
3.3.6 System interfacing
3.3.7 Quality issues
3.3.8 System modifications
3.3.9 Physical environment
3.3.10 Security issues
3.3.11 Resources and management issues

Nonfunctional Requirements (Questions to overcome “Writers block”)

User interface and human factors

- ✓ What type of user will be using the system?
- ✓ Will more than one type of user be using the system?
- ✓ What training will be required for each type of user?
- ✓Is it important that the system is easy to learn?
- ✓Should users be protected from making errors?
- ✓What input/output devices are available

Documentation

- ✓What kind of documentation is required?
- ✓What audience is to be addressed by each document?

Nonfunctional Requirements (2)

- Hardware considerations

✓ What hardware is the proposed system to be used on?

✓What are the characteristics of the target hardware,

✓including memory size and auxiliary storage space?

- Performance characteristics

✓Are there speed, throughput, response time constraints on the system?

✓ Are there size or capacity constraints on the data to be processed by the system?

- Error handling and extreme conditions

✓How should the system respond to input errors?

✓How should the system respond to extreme conditions?

Nonfunctional Requirements (3)

- System interfacing

Is input coming from systems outside the proposed system?

Is output going to systems outside the proposed system?

Are there restrictions on the format or medium that must be used for input or output?

- Quality issues

What are the requirements for reliability?

Must the system trap faults?

What is the time for restarting the system after a failure?

Is there an acceptable downtime per 24-hour period?

Is it important that the system be portable?

Nonfunctional Requirements (4)

- System Modifications

What parts of the system are likely to be modified?

What sorts of modifications are expected?

- Physical Environment

Where will the target equipment operate?

Is the target equipment in one or several locations?

Will the environmental conditions be ordinary?

- Security Issues

Must access to data or the system be controlled?

Is physical security an issue?

Nonfunctional Requirements (5)

- Resources and Management Issues

How often will the system be backed up?

Who will be responsible for the back up?

Who is responsible for system installation?

Who will be responsible for system maintenance?

Thank you!!!!